

Carina

Theory Manual

Formulation, constitutive models, and solution algorithms

Alejandro Mota

September 24, 2026

About this manual

Carina's documentation is split by the question it answers. The user manual (`docs/`, built with Documenter) answers *what do I write in the input deck*. The benchmark report (`benchmark_report.md`) answers *how fast is it, and on what hardware*. This manual answers *why the algorithms work*.

It is written to be read alongside the source. Each algorithm is stated in the form the code actually implements — not the form most convenient to typeset — and the margin of each section names the file implementing it. Where the implementation departs from a textbook statement, the departure is the point of the section rather than a footnote to it: those departures are usually the result of a measurement, and the measurement is cited.

Nothing here is a substitute for reading the code, and the code is the authority wherever the two disagree. If you find such a disagreement, it is a defect in this manual.

Contents

About this manual	i
1 Notation	1
2 Formulation	3
2.1 Strong and weak form	3
2.2 Discretization	3
2.3 Consistent tangent	4
2.4 Dirichlet conditions by elimination	4
2.5 Mass matrix and lumping	5
3 Constitutive models	6
3.1 Linear elastic	6
3.2 Neo-Hookean	6
3.3 Hencky	6
3.4 Finite-deformation J_2 plasticity	7
3.5 Tangent strategies	7
4 Time integration	9
4.1 Quasi-static	9
4.2 Newmark- β	9
4.3 Central difference	10
4.4 Adaptive time stepping	11
5 Newton's method	12
5.1 The iteration	12
5.2 Globalization by line search	12
5.3 Status tests	13
6 Inexact Newton and forcing terms	14
6.1 The over-solving problem	14
6.2 Eisenstat–Walker choice 2	14
6.3 Three safeguards	15
6.4 Choosing $\eta_{\max}$	16
6.5 What the method buys	17
6.6 Interaction with lagged preconditioners	17

7	Quasi-Newton and gradient methods	18
7.1	L-BFGS	18
7.2	Nonlinear conjugate gradient	19
7.3	Preconditioned steepest descent	19
8	Krylov methods	20
8.1	Why conjugate gradients	20
8.2	Preconditioned conjugate gradients	20
8.3	Device-resident conjugate gradients	21
8.4	Direct solvers	21
9	Preconditioners	22
9.1	Jacobi	22
9.2	Incomplete LDL^T	22
9.3	Chebyshev polynomial preconditioner	22
10	Smoothed-aggregation multigrid	24
10.1	Why multigrid	24
10.2	The near-nullspace	24
10.3	Hierarchy construction	25
10.4	The V-cycle	26
10.5	GPU execution	26
10.6	Lagging and staleness	26
10.7	When multigrid is the right tool	27
11	Matrix-free operators	28
11.1	The action, not the matrix	28
11.2	Operator fusion	28
11.3	Where the time goes	29
11.4	Two-phase assembly	29
11.5	Mixed precision	29

Chapter 1

Notation

Bold upright capitals denote second-order tensors and matrices, bold lowercase denotes vectors, and blackboard bold denotes fourth-order tensors. Bold Greek follows the same convention through the `boldtensors` package: $\boldsymbol{\sigma}$ is the Cauchy stress tensor, σ a scalar.

Continuum quantities

Symbol	Meaning
$\boldsymbol{X}, \boldsymbol{x}$	material and spatial position
$\boldsymbol{u}$	displacement field, $\boldsymbol{x} = \boldsymbol{X} + \boldsymbol{u}$
$\boldsymbol{F} = \boldsymbol{I} + \nabla \boldsymbol{u}$	deformation gradient
$J = \det \boldsymbol{F}$	Jacobian of the deformation
$\boldsymbol{C} = \boldsymbol{F}^\top \boldsymbol{F}$	right Cauchy–Green tensor
$\boldsymbol{P}$	first Piola–Kirchhoff stress
$\boldsymbol{\sigma}, \boldsymbol{\tau} = J \boldsymbol{\sigma}$	Cauchy and Kirchhoff stress
Ψ	strain-energy density per unit reference volume
$\mathbb{A} = \partial \boldsymbol{P} / \partial \nabla \boldsymbol{u}$	material tangent
ρ_0	reference mass density

Discrete quantities

Symbol	Meaning
n	number of unknown (free) degrees of freedom
$\mathbf{u}, \mathbf{v}, \mathbf{a}$	nodal displacement, velocity, acceleration
$\mathbf{R}$	residual (out-of-balance force)
$\mathbf{K} = \partial \mathbf{R} / \partial \mathbf{u}$	tangent stiffness
$\mathbf{M}$	mass matrix
$\mathbf{K}_{\text{eff}}$	effective operator of an implicit step
c_M	mass coefficient of $\mathbf{K}_{\text{eff}}$, Section 4.2
$\mathbf{D} = \text{diag}(\mathbf{K})$	diagonal of an operator
$\mathbf{M}$	preconditioner, $\mathbf{M} \approx \mathbf{K}^{-1}$
$\mathbf{P}, \mathbf{R}$	prolongation and restriction
$\mathbf{B}$	near-nullspace, Section 10.2
η_k	forcing term at Newton iteration k , Chapter 6
$\lambda_{\max}$	largest eigenvalue of $\mathbf{D}^{-1} \mathbf{K}$

Index conventions

Subscript k indexes nonlinear (Newton) iterations, subscript i or j indexes linear (Krylov) iterations, subscript n indexes time steps, and superscript ℓ indexes multigrid levels. A quantity written without a time subscript is understood at t_{n+1} , the step being solved.

Norms are Euclidean unless a subscript says otherwise; $\|\mathbf{r}\|_{\mathbf{M}} = \sqrt{\mathbf{r}^T \mathbf{M} \mathbf{r}}$ denotes the norm induced by a preconditioner, which is the quantity preconditioned conjugate gradients actually monitors (Section 8.2).

Free and constrained degrees of freedom

Carina imposes Dirichlet conditions by *elimination*, never by penalty. The global degree-of-freedom set partitions into unknowns and prescribed values, and every operator named above is understood as the unknown–unknown block unless stated otherwise. Chapter 2 makes this precise and Section 2.4 explains why the choice matters for every Krylov method in this manual.

Chapter 2

Formulation

Implementation: `src/physics.jl`

2.1 Strong and weak form

Carina solves finite-deformation solid mechanics in the reference configuration. Linear momentum balance reads

$$\nabla \cdot \mathbf{P} + \rho_0 \mathbf{b} = \rho_0 \ddot{\mathbf{u}}, \quad (2.1)$$

with $\mathbf{P}$ the first Piola–Kirchhoff stress, $\mathbf{b}$ a body force per unit mass, and ρ_0 the reference density. Multiplying by an admissible variation $\delta \mathbf{u}$ vanishing on the Dirichlet boundary Γ_u and integrating by parts gives the weak form: find $\mathbf{u}$ with $\mathbf{u} = \bar{\mathbf{u}}$ on Γ_u such that

$$\int_{\Omega_0} \mathbf{P} : \nabla \delta \mathbf{u} \, dV + \int_{\Omega_0} \rho_0 \ddot{\mathbf{u}} \cdot \delta \mathbf{u} \, dV = \int_{\Omega_0} \rho_0 \mathbf{b} \cdot \delta \mathbf{u} \, dV + \int_{\Gamma_t} \bar{\mathbf{t}} \cdot \delta \mathbf{u} \, dA \quad (2.2)$$

for all admissible $\delta \mathbf{u}$, where $\bar{\mathbf{t}}$ is the prescribed traction on Γ_t .

The kinematics enter only through the deformation gradient

$$\mathbf{F} = \mathbf{I} + \nabla \mathbf{u}, \quad (2.3)$$

and the constitutive response enters only through $\mathbf{P}(\mathbf{F})$. This is the entire coupling surface between Chapter 3 and everything downstream: a constitutive model supplies $\mathbf{P}$ given $\nabla \mathbf{u}$, and optionally its derivative.

2.2 Discretization

Introducing isoparametric shape functions N_a over an element mesh and writing $\mathbf{u} \approx \sum_a N_a \mathbf{u}_a$, (2.2) becomes the semi-discrete system

$$\mathbf{M} \mathbf{a} + \mathbf{f}^{\text{int}}(\mathbf{u}) = \mathbf{f}^{\text{ext}}(t), \quad (2.4)$$

with

$$\mathbf{M}_{ab} = \int_{\Omega_0} \rho_0 N_a N_b \, dV, \quad \mathbf{f}_a^{\text{int}}(\mathbf{u}) = \int_{\Omega_0} \mathbf{P}(\mathbf{F}) \cdot \nabla N_a \, dV. \quad (2.5)$$

Both integrals are evaluated by Gauss quadrature element by element. The residual that every solver in this manual drives to zero is

$$\mathbf{R}(\mathbf{u}) = \mathbf{f}^{\text{ext}} - \mathbf{f}^{\text{int}}(\mathbf{u}) - \mathbf{M}\mathbf{a}, \quad (2.6)$$

with the inertial term absent in the quasi-static case (Section 4.1).

Remark 2.1. Carina stores $\mathbf{R}$ already negated relative to the force residual, so the Newton system is $\mathbf{K} \Delta \mathbf{u} = \mathbf{R}$ with a plus sign throughout the implementation. The sign convention is stated in `linear_solvers.jl` at the dispatch point and is followed uniformly here.

2.3 Consistent tangent

Differentiating (2.6) with respect to the nodal displacements gives the tangent stiffness

$$\mathbf{K}_{ab} = \frac{\partial \mathbf{f}_a^{\text{int}}}{\partial \mathbf{u}_b} = \int_{\Omega_0} \nabla N_a \cdot \mathbb{A} \cdot \nabla N_b \, dV, \quad \mathbb{A} = \frac{\partial \mathbf{P}}{\partial \nabla \mathbf{u}}. \quad (2.7)$$

The fourth-order material tangent $\mathbb{A}$ is the only object a constitutive model must supply beyond the stress itself; how Carina obtains it — in closed form, by automatic differentiation, or by finite differences — is the subject of Section 3.5.

2.4 Dirichlet conditions by elimination

Prescribed displacements are imposed by removing the corresponding equations and unknowns, not by adding a penalty term or a Lagrange multiplier. Partitioning the degrees of freedom into unknowns $\mathbf{f}$ and prescribed values $\mathbf{p}$,

$$\begin{bmatrix} \mathbf{K}_{\text{ff}} & \mathbf{K}_{\text{fp}} \\ \mathbf{K}_{\text{pf}} & \mathbf{K}_{\text{pp}} \end{bmatrix} \begin{bmatrix} \Delta \mathbf{u}_{\text{f}} \\ \Delta \mathbf{u}_{\text{p}} \end{bmatrix} = \begin{bmatrix} \mathbf{R}_{\text{f}} \\ \mathbf{R}_{\text{p}} \end{bmatrix}, \quad (2.8)$$

only the first block row is solved, with $\Delta \mathbf{u}_{\text{p}}$ known. The cross term $\mathbf{K}_{\text{fp}} \Delta \mathbf{u}_{\text{p}}$ is carried automatically because the element kernels read displacement fields that already hold the prescribed values at constrained nodes; nothing assembles $\mathbf{K}_{\text{fp}}$ explicitly.

This choice is not cosmetic. A penalty imposition multiplies selected diagonal entries by a large factor — 10^{15} is typical — which raises the condition number of the operator by the same factor and destroys the convergence rate of every Krylov method in Chapter 8. The eliminated system carries no artificial conditioning at all, so the iteration counts reported throughout this manual reflect the physics of the problem rather than the bookkeeping of the boundary conditions.

Remark 2.2. One place in Carina does see penalty-scale entries: the assembled Chebyshev path, which forms $\mathbf{K}$ including constrained rows before scaling. That is why the polynomial there is built on the symmetrically scaled operator $\mathbf{D}^{-1/2} \mathbf{K} \mathbf{D}^{-1/2}$, whose eigenvalues are $O(1)$ even when $\mathbf{K}$'s are not. See Section 9.3.

2.5 Mass matrix and lumping

The consistent mass of (2.4) is used by the implicit dynamic integrator. The explicit integrator instead uses the row-sum lumped mass

$$M_{aa}^L = \sum_b M_{ab}, \quad (2.9)$$

which is diagonal and therefore invertible in $O(n)$ with no linear solve at all — the property that makes explicit integration attractive (Section 4.3).

Chapter 3

Constitutive models

Implementation: `src/physics.jl`, `src/materials.jl`

Carina delegates constitutive evaluation to `ConstitutiveModels.jl`. The interface is narrow by design: a model returns $\mathbf{P}$ given $\nabla \mathbf{u}$ and, for models carrying internal variables, an updated state. Four models are wired into the input parser.

3.1 Linear elastic

Small-strain isotropic elasticity, $\boldsymbol{\sigma} = \lambda \text{tr}(\boldsymbol{\varepsilon})\mathbf{I} + 2\mu\boldsymbol{\varepsilon}$ with $\boldsymbol{\varepsilon} = \frac{1}{2}(\nabla \mathbf{u} + \nabla \mathbf{u}^\top)$. Its tangent is constant, which Carina exploits: for a linear-elastic model the stiffness is assembled once and the factorization — or, on GPU, the Cholesky factor uploaded for triangular solves (Section 8.4) — is cached for the entire run.

3.2 Neo-Hookean

The compressible neo-Hookean strain energy

$$\Psi(\mathbf{F}) = \frac{\mu}{2} (\text{tr } \mathbf{C} - 3) - \mu \ln J + \frac{\lambda}{2} (\ln J)^2 \quad (3.1)$$

gives

$$\mathbf{P} = \mu (\mathbf{F} - \mathbf{F}^{-\top}) + \lambda \ln J \mathbf{F}^{-\top}. \quad (3.2)$$

This is the workhorse for the benchmark problems throughout the performance campaigns, and the one model whose tangent *action* Carina writes in closed form (Section 3.5).

3.3 Hencky

Logarithmic (true) strain elasticity, $\boldsymbol{\tau} = \lambda \text{tr}(\mathbf{h})\mathbf{I} + 2\mu\mathbf{h}$ with $\mathbf{h} = \frac{1}{2} \ln \mathbf{C}$. The matrix logarithm is evaluated by inverse scaling-and-squaring with Padé approximants rather than by spectral decomposition: the spectral route loses accuracy and differentiability when eigenvalues of $\mathbf{C}$ coalesce, which is exactly the neighborhood of the undeformed state.

3.4 Finite-deformation J_2 plasticity

Rate-independent J_2 flow with linear isotropic hardening, integrated by a radial-return map at each quadrature point. The trial elastic state is computed, and if the yield function

$$f = \left\| \text{dev } \boldsymbol{\tau}^{\text{tr}} \right\| - \sqrt{\frac{2}{3}} (\sigma_y + H \bar{\varepsilon}^p) \quad (3.3)$$

is positive the plastic multiplier is solved for and the stress projected back onto the yield surface. The model carries internal state — the plastic strain tensor and the equivalent plastic strain — which makes it *path dependent*: the same displacement reached by different increments gives different stresses. This has two consequences that surface elsewhere in this manual. Large load steps miss the yield surface and must be prevented by adaptive stepping (Section 4.4); and the return map can raise domain errors on non-physical trial states, which the solver treats as a step failure rather than a crash.

3.5 Tangent strategies

Equation (2.7) needs $\mathbb{A} = \partial \mathbf{P} / \partial \nabla \mathbf{u}$ at every quadrature point. Carina obtains it three different ways, chosen by what the model can support.

Closed form. For neo-Hookean the directional derivative is written analytically. With $\delta \mathbf{F} = \nabla \mathbf{v}$, and abbreviating $a = \text{tr}(\mathbf{F}^{-1} \delta \mathbf{F})$ and $\mathbf{W} = \mathbf{F}^{-\top} \delta \mathbf{F}^\top \mathbf{F}^{-\top}$,

$$\delta \mathbf{P} = \mu (\delta \mathbf{F} + \mathbf{W}) + \lambda a \mathbf{F}^{-\top} - \lambda \ln J \mathbf{W}. \quad (3.4)$$

This form is what the matrix-free operator of Chapter 11 evaluates: it needs the tangent's action on *one* direction, never the full 81-component $\mathbb{A}$, so the closed form costs a handful of 3×3 products instead of a rank-four object.

Automatic differentiation. For stateless hyperelastic models generally, $\mathbb{A}$ is obtained by differentiating `pk1_stress` through dual numbers. This is exact to machine precision and requires nothing from the model author.

Finite differences. Models carrying state — J_2 plasticity — run a return map inside `pk1_stress`, whose branches are not differentiable in the ordinary sense. Carina falls back to a forward finite difference over the nine components of $\nabla \mathbf{u}$,

$$\mathbb{A}_{ijkl} \approx \frac{\mathbf{P}_{ij}(\nabla \mathbf{u} + h_{kl} \mathbf{e}_k \otimes \mathbf{e}_l) - \mathbf{P}_{ij}(\nabla \mathbf{u})}{h_{kl}}, \quad (3.5)$$

at a cost of nine stress evaluations per quadrature point. The perturbation is adaptive,

$$h_{kl} = \max(\sqrt{\varepsilon_{\text{mach}}} |(\nabla \mathbf{u})_{kl}|, h_{\min}), \quad h_{\min} = 10^{-10}, \quad (3.6)$$

following the Sierra/SM convention. The relative term dominates wherever the strain is resolved; the floor prevents a vanishing perturbation — and hence catastrophic cancellation — at components that happen to be near zero, which in an undeformed region is most of them.

Remark 3.1. The finite-difference tangent is a *tangent*, not the residual: it enters only the Newton direction and the preconditioner, never the convergence test. An approximate tangent slows Newton's asymptotic rate but cannot change the solution it converges to, because that is fixed by $\mathbf{R} = \mathbf{0}$. The same argument licenses the reduced-precision operator of Section 11.5.

Chapter 4

Time integration

Implementation: src/integrators.jl

Three integrators share one `predict!` / `evaluate!` / `correct!` protocol, so the nonlinear and linear machinery downstream is written once.

4.1 Quasi-static

Inertia is dropped from (2.4) and the load is applied in pseudo-time increments:

$$\mathbf{R}(\mathbf{u}) = \mathbf{f}^{\text{ext}}(t_{n+1}) - \mathbf{f}^{\text{int}}(\mathbf{u}) = \mathbf{0}, \quad \mathbf{K} = \frac{\partial \mathbf{f}^{\text{int}}}{\partial \mathbf{u}}. \quad (4.1)$$

The operator is the bare tangent stiffness. For a well-shaped mesh its condition number grows like $(L/h)^2$ with the mesh refinement ratio, which is what makes quasi-static problems the natural home for multigrid (Chapter 10) and the reason a diagonal preconditioner degrades so visibly with problem size.

4.2 Newmark- β

The Newmark family parameterizes the step by β and γ :

$$\mathbf{u}_{n+1} = \mathbf{u}_n + \Delta t \mathbf{v}_n + \Delta t^2 \left[\left(\frac{1}{2} - \beta \right) \mathbf{a}_n + \beta \mathbf{a}_{n+1} \right], \quad (4.2)$$

$$\mathbf{v}_{n+1} = \mathbf{v}_n + \Delta t [(1 - \gamma) \mathbf{a}_n + \gamma \mathbf{a}_{n+1}]. \quad (4.3)$$

Carina implements this in predictor–corrector form. The predictor collects everything known at t_n ,

$$\mathbf{u}^{\text{pred}} = \mathbf{u}_n + \Delta t \mathbf{v}_n + \Delta t^2 \left(\frac{1}{2} - \beta \right) \mathbf{a}_n, \quad \mathbf{v}^{\text{pred}} = \mathbf{v}_n + \Delta t (1 - \gamma) \mathbf{a}_n, \quad (4.4)$$

after which the acceleration follows from the displacement alone:

$$\mathbf{a}_{n+1} = c_M \left(\mathbf{u}_{n+1} - \mathbf{u}^{\text{pred}} \right), \quad \boxed{c_M = \frac{1}{\beta \Delta t^2}} \quad (4.5)$$

and the corrector restores the velocity, $\mathbf{v}_{n+1} = \mathbf{v}^{\text{pred}} + \gamma \Delta t \mathbf{a}_{n+1}$. Displacement is thus the only unknown, and the Newton system for it has the *effective operator*

$$\mathbf{K}_{\text{eff}} = \mathbf{K} + c_M \mathbf{M}. \quad (4.6)$$

Equation (4.5) governs almost everything about how implicit dynamics behaves numerically, and it is worth being explicit about why. As Δt shrinks, $c_M \sim \Delta t^{-2}$ grows without bound and $\mathbf{K}_{\text{eff}}$ becomes mass-dominated. The mass matrix is well conditioned — it is a Gram matrix of shape functions, independent of the deformation — so $\mathbf{K}_{\text{eff}}$ inherits that conditioning. Three consequences recur throughout this manual:

- A diagonal (Jacobi) preconditioner is unusually effective at small Δt , because $\text{diag}(c_M \mathbf{M})$ captures the dominant term.
- Multigrid, whose value is mesh-independent iteration counts on a stiffness-dominated operator, has little left to remove; measurements confirm it does not repay its per-application cost in this regime (Section 10.7).
- A low-rank quasi-Newton model of $\mathbf{K}_{\text{eff}}^{-1}$ is viable at small Δt and not at all in the quasi-static limit (Section 7.1).

Constrained degrees of freedom. The predictor is engineered so that (4.5) reproduces the prescribed acceleration exactly at constrained nodes: $\mathbf{u}_p^{\text{pred}} = \mathbf{u}_p - \ddot{\mathbf{g}}(t_{n+1})/c_M$, whence $\mathbf{a}_p = \ddot{\mathbf{g}}(t_{n+1})$. This keeps the inertial cross-term $\mathbf{M}_{\text{fp}} \ddot{\mathbf{g}}$ in the residual without special-casing it, and it is why the integrator carries full-length $\mathbf{u}$, $\mathbf{v}$, $\mathbf{a}$ arrays rather than free-DOF slices.

Stability. $\beta = \frac{1}{4}$, $\gamma = \frac{1}{2}$ (average acceleration) is unconditionally stable and second-order accurate, and is the default. Numerical dissipation, when wanted, is available through the HHT- α generalization the integrator carries.

4.3 Central difference

The explicit integrator advances

$$\mathbf{u}_{n+1} = \mathbf{u}_n + \Delta t \mathbf{v}_n + \frac{1}{2} \Delta t^2 \mathbf{a}_n, \quad \mathbf{a}_{n+1} = \left(\mathbf{M}^L \right)^{-1} \mathbf{R}(\mathbf{u}_{n+1}), \quad (4.7)$$

followed by $\mathbf{v}_{n+1} = \mathbf{v}_n + \Delta t [(1 - \gamma) \mathbf{a}_n + \gamma \mathbf{a}_{n+1}]$. With a lumped mass the acceleration update is a component-wise division: no linear solve, no nonlinear solve, no tangent. Nothing in Chapters 5–10 applies, which is why the input parser ignores the `solver` block entirely for this integrator.

Critical time step. Stability is conditional on the CFL limit

$$\Delta t \leq \text{CFL} \cdot \min_e \frac{\ell_e}{c_e}, \quad c_e = \sqrt{\frac{\lambda + 2\mu}{\rho_0}}, \quad (4.8)$$

with ℓ_e a characteristic element length and c_e the dilatational wave speed. Carina recomputes this bound periodically during the run rather than once at setup, because ℓ_e shrinks as elements compress.

4.4 Adaptive time stepping

Both implicit integrators shrink Δt by a decrease factor when the nonlinear solve fails and grow it by an increase factor, up to a ceiling, when it succeeds. A failure is any of: the status tree reporting FAILED (Section 5.3), a non-finite residual, or a domain error raised inside a constitutive model.

The mechanism matters most for J_2 plasticity, where it is not merely an efficiency device but a correctness one. Because the return map is path dependent, a step large enough to traverse the yield surface in one increment integrates the wrong path — and converges perfectly well to the wrong answer. Step-size control plus the line search of Section 5.2 is what keeps the increments small enough for the return map to track the true path.

Chapter 5

Newton's method

Implementation: `src/nonlinear_solvers.jl`

5.1 The iteration

Newton–Raphson linearizes $\mathbf{R}(\mathbf{u}) = \mathbf{0}$ about the current iterate:

$$\mathbf{K}(\mathbf{u}_k) \Delta \mathbf{u}_k = \mathbf{R}(\mathbf{u}_k), \quad \mathbf{u}_{k+1} = \mathbf{u}_k + \alpha_k \Delta \mathbf{u}_k, \quad (5.1)$$

with $\mathbf{K}$ the tangent (2.7) for quasi-static problems and $\mathbf{K}_{\text{eff}}$ of (4.6) for implicit dynamics. With an exact tangent, an exact linear solve, and $\alpha_k = 1$, convergence near the solution is quadratic. Each of those three assumptions is relaxed somewhere in Carina, and each relaxation is the subject of a later chapter: approximate tangents in Section 3.5, inexact linear solves in Chapter 6, and $\alpha_k < 1$ immediately below.

5.2 Globalization by line search

Far from the solution the full Newton step can be worse than no step at all. At large Δt or a large load increment the predictor may sit far enough from equilibrium that $\mathbf{u}_k + \Delta \mathbf{u}_k$ inverts elements, producing $J \leq 0$ and a non-finite residual.

Carina globalizes with Armijo backtracking on the merit function

$$m(\alpha) = \frac{1}{2} \|\mathbf{R}(\mathbf{u}_k + \alpha \Delta \mathbf{u}_k)\|^2, \quad (5.2)$$

accepting the first $\alpha \in \{1, \tau, \tau^2, \dots\}$ satisfying the sufficient-decrease condition

$$m(\alpha) \leq (1 - 2c\alpha) m(0), \quad (5.3)$$

with backtrack factor $\tau = \frac{1}{2}$ and $c = 10^{-4}$ by default. Equation (5.3) is the standard Armijo condition specialized to this merit function: since $\Delta \mathbf{u}_k$ solves $\mathbf{K} \Delta \mathbf{u} = \mathbf{R}$, the directional derivative is $m'(0) = -\|\mathbf{R}\|^2 = -2m(0)$, and substituting into $m(\alpha) \leq m(0) + c\alpha m'(0)$ gives (5.3) directly.

The line search is *on by default*. It costs roughly one extra residual evaluation per iteration when the full step is already good, since $\alpha = 1$ is then accepted immediately, and it is the only thing standing between a bad predictor and an inverted element.

Algorithm 1 Newton step with Armijo backtracking

Input: iterate $\mathbf{u}_k$, residual $\mathbf{R}_k$, tangent $\mathbf{K}_k$, parameters c, τ , budget $j_{\max}$

- 1: $\eta_k \leftarrow \text{FORCINGTERM}(\cdot)$ $\triangleright$ *Algorithm 2; $\eta_k = \varepsilon_{\text{lin}}$ if disabled*
- 2: solve $\mathbf{K}_k \Delta \mathbf{u}_k = \mathbf{R}_k$ to relative tolerance η_k
- 3: $m_0 \leftarrow \frac{1}{2} \|\mathbf{R}_k\|^2$, $\alpha \leftarrow 1$
- 4: **for** $j = 1$ **to** $j_{\max}$ **do**
- 5: evaluate $\mathbf{R}$ at $\mathbf{u}_k + \alpha \Delta \mathbf{u}_k$; $m \leftarrow \frac{1}{2} \|\mathbf{R}\|^2$
- 6: **if** $m \leq (1 - 2c\alpha) m_0$ **then**
- 7: **return** $\mathbf{u}_k + \alpha \Delta \mathbf{u}_k$ $\triangleright$ *Armijo satisfied*
- 8: **end if**
- 9: $\alpha \leftarrow \tau \alpha$
- 10: **end for**
- 11: restore the pre-step material state $\triangleright$ *trials advanced it; see Remark 5.1*
- 12: **return** $\mathbf{u}_k + \Delta \mathbf{u}_k$ $\triangleright$ *budget exhausted; outer failure machinery takes over*

Remark 5.1. If the backtracking budget is exhausted without satisfying (5.3), Carina restores the pre-step material state and accepts the full step, leaving the outer failure machinery of Section 4.4 to handle it. Restoring the state matters: the trial evaluations have already advanced the internal variables of a path-dependent model, and accepting a step on top of a state advanced by a rejected trial would silently corrupt the history.

Remark 5.2. An inexact Newton direction (Chapter 6) remains a descent direction for m as long as the linear residual is smaller than $\|\mathbf{R}\|$ — that is, whenever the forcing term satisfies $\eta_k < 1$, which the clamp of Section 6.3 enforces. The line search therefore composes with inexact solves without modification.

5.3 Status tests

Termination is a *tree* of composable tests rather than a fixed criterion, following the NOX pattern. Leaves test the residual ($\|\mathbf{R}\| < \varepsilon_a$ or $\|\mathbf{R}\| / \|\mathbf{R}_0\| < \varepsilon_r$), the update ($\|\Delta \mathbf{u}\|$, absolute or relative to $\|\mathbf{u}\|$), the iteration count, finiteness, divergence, or stagnation; interior nodes combine them with AND/OR.

Two properties of the combination rules are worth stating because algorithms elsewhere depend on them. In an OR node, CONVERGED beats FAILED when both occur on the same iteration. And a *finite-value* test is appended to every tree automatically, so a NAN residual always terminates the solve regardless of what the user wrote.

The tree is not merely a convenience. Section 6.3 needs the residual norm the solve is actually aiming at, and reads it off this tree — taking the loosest child of an OR and the tightest of an AND — rather than from a nominal tolerance that a user-supplied tree may have overridden.

Chapter 6

Inexact Newton and forcing terms

Implementation: `src/linear_solvers.jl`

6.1 The over-solving problem

Newton's step (5.1) is a linear system, and when it is solved by an iterative method the accuracy of that solve is a free parameter. An *inexact Newton method* accepts any $\Delta \mathbf{u}_k$ satisfying

$$\|\mathbf{R}(\mathbf{u}_k) - \mathbf{K}(\mathbf{u}_k)\Delta \mathbf{u}_k\| \leq \eta_k \|\mathbf{R}(\mathbf{u}_k)\|, \quad (6.1)$$

where $\eta_k \in [0, 1)$ is the *forcing term*. Choosing η_k constant and small — the naive choice, and Carina's original behavior — solves every linear system to the same tolerance regardless of how far the nonlinear iteration still has to go.

That this wastes work is not an abstract concern. On a 530,523-DOF Newmark step, conjugate gradients spent 181, 205, and 193 iterations on the three solves of a step while the nonlinear residual fell

$$1.61 \times 10^4 \longrightarrow 1.34 \times 10^2 \longrightarrow 7.79 \times 10^{-2} \longrightarrow 4.60 \times 10^{-8}.$$

The first two steps were computed to eight digits of linear accuracy and then discarded by the next Newton correction. The pattern on a quasi-static J_2 specimen is worse: roughly 350 iterations on each of the first seven solves while the residual crawls from 1.57×10^3 to 1.76×10^2 , after which the line search backtracks to $\alpha = \frac{1}{4}$ and throws away most of the step it just paid for.

6.2 Eisenstat–Walker choice 2

The remedy is to tie η_k to the convergence actually being observed. Carina implements choice 2 of Eisenstat and Walker [1]:

$$\eta_k^{\text{EW}} = \gamma \left(\frac{\|\mathbf{R}_k\|}{\|\mathbf{R}_{k-1}\|} \right)^\alpha, \quad \gamma \in (0, 1], \quad \alpha \in (1, 2]. \quad (6.2)$$

The reasoning is that the ratio of successive residual norms estimates how well the linear model predicted the nonlinear decrease. When the previous step behaved as the model said it would,

the ratio is small and a tighter solve is warranted; when the model is doing badly, a tight solve is refining a direction that is wrong anyway. The default $\alpha = (1 + \sqrt{5})/2$ is the paper's, and recovers superlinear convergence of the outer iteration.

On the first iteration of a step no ratio exists, and η_0 is taken from the `initial` parameter.

6.3 Three safeguards

Equation (6.2) alone is not usable. Carina composes it with three guards, applied in order.

1. Against η collapsing too fast. A single unusually good step can drive (6.2) to a very small value that the local convergence rate does not justify, wasting on the next solve exactly what the method exists to save. Eisenstat and Walker's own safeguard raises η back toward its predecessor whenever that predecessor was itself large:

$$\eta_k \leftarrow \max\left(\eta_k^{\text{EW}}, \gamma \eta_{k-1}^\alpha\right) \quad \text{whenever} \quad \gamma \eta_{k-1}^\alpha > 0.1. \quad (6.3)$$

The threshold 0.1 is the paper's and is not a tuning parameter in Carina. It nonetheless has a consequence for the parameter that *is* tunable, which Section 6.4 takes up.

2. Against over-solving the final step. Near convergence the ratio in (6.2) becomes tiny and demands far more linear accuracy than the nonlinear stopping test will ever reward. Kelley's guard [2] bounds η from below by what the target actually requires:

$$\eta_k \leftarrow \max\left(\eta_k, \frac{s \tau}{\|\mathbf{R}_k\|}\right), \quad (6.4)$$

where τ is the residual norm the nonlinear test is aiming at and s a safety factor, $\frac{1}{2}$ by default. Carina reads τ from the status test tree of Section 5.3, so a deck's own `termination` block governs rather than a nominal tolerance it may have overridden.

3. A hard clamp. Finally

$$\eta_k \leftarrow \min\left(\max(\eta_k, \varepsilon_{\text{lin}}), \eta_{\text{max}}\right), \quad (6.5)$$

where ε_{lin} is the deck's own linear tolerance.

The order matters, and Algorithm 2 fixes it: the two lower bounds are applied before the clamp, so neither can push η outside $[\varepsilon_{\text{lin}}, \eta_{\text{max}}]$.

Proposition 6.1 (Safety). *Under (6.5), a forcing term can only loosen a linear solve relative to the fixed-tolerance baseline, never tighten one.*

This is the property that makes the method switchable on without re-validating a model. The lower clamp guarantees the final Newton iterations run at exactly the tolerance the user requested, so the converged solution is as accurate as it was before and an A/B comparison against a fixed tolerance compares like with like. Measurements bear this out: across CPU and A100 runs in both implicit regimes, $\|\mathbf{u}\|_\infty$ agrees with the fixed-tolerance baseline to every printed digit at every output stop.

Algorithm 2 Forcing term at Newton iteration k

Input: $\|\mathbf{R}_k\|$; previous $\|\mathbf{R}_{k-1}\|$ and η_{k-1} if any; target τ ; parameters $\gamma, \alpha, \eta_{\max}, \eta_0, s$; linear tolerance ε_{lin}

```

1: if no previous residual then
2:    $\eta \leftarrow \eta_0$   $\triangleright$  first iteration of a step
3: else
4:    $\eta \leftarrow \gamma (\|\mathbf{R}_k\| / \|\mathbf{R}_{k-1}\|)^\alpha$ 
5:   if  $\gamma \eta_{k-1}^\alpha > 0.1$  then
6:      $\eta \leftarrow \max(\eta, \gamma \eta_{k-1}^\alpha)$   $\triangleright$  safeguard 1: anti-collapse
7:   end if
8: end if
9: if  $s > 0$  and  $\|\mathbf{R}_k\| > 0$  then
10:   $\eta \leftarrow \max(\eta, s\tau / \|\mathbf{R}_k\|)$   $\triangleright$  safeguard 2: over-solve guard
11: end if
12:  $\eta_k \leftarrow \min(\max(\eta, \varepsilon_{\text{lin}}), \eta_{\max})$   $\triangleright$  safeguard 3: clamp
13: return  $\eta_k$ 

```

Remark 6.1. Preconditioned conjugate gradients stops on the preconditioned residual $\mathbf{M}$ -norm relative to its own initial value, not on the unpreconditioned ratio of (6.1) for which the theory is stated. The forcing term is a heuristic in either norm; Proposition 6.1 is what bounds the damage when the two disagree.

6.4 Choosing $\eta_{\max}$

The literature’s default is $\eta_{\max} = 0.9$. Carina’s is 0.2, and the reason is structural rather than empirical fitting.

Measurements across a CPU quasi-static sweep and two A100 regimes show that the *total* linear work is nearly invariant in $\eta_{\max}$: 63.1k, 62.9k and 63.6k CG iterations at $\eta_{\max} = 0.1, 0.2, 0.5$ respectively on the J_2 specimen, and 1019 versus 1011 on the A100 Newmark problem. What $\eta_{\max}$ actually controls is how many *Newton* iterations are spent buying that work — 413 versus 547 on the same CPU sweep.

The mechanism is safeguard (6.3), which activates when $\gamma\eta^\alpha > 0.1$. At the default α :

$\eta_{\max}$	$\eta_{\max}^\alpha$	Behavior
0.2	0.076	below the knee; the residual ratio drives η
0.5	0.326	above it; the safeguard pins η high for several iterations
0.9	0.842	far above

So 0.2 is the largest round value that keeps Eisenstat–Walker’s anti-collapse safeguard out of the way of Eisenstat–Walker’s own adaptivity. Two different problems on two different architectures selected it independently.

6.5 What the method buys

A forcing term trades linear work for nonlinear work. It therefore pays in proportion to how much of a step is the linear solve — most on the matrix-free GPU paths of Chapter 11, where the step essentially *is* the stiffness matrix–vector product, and least on assembled CPU runs where residual assembly and the line search already dominate. Measured with the defaults:

Configuration	CG iterations	Per-step wall
CPU quasi-static, J_2 , 57k DOF	$2.47\times$ fewer	$1.30\times$
A100 Newmark, 530k DOF, CG + Jacobi	$2.27\times$ fewer	$1.45\times$
A100 quasi-static, 530k DOF, CG + AMG	$2.05\times$ fewer	$1.59\times$

The quasi-static figure is on top of the roughly twofold advantage algebraic multigrid already holds over Jacobi on that problem.

6.6 Interaction with lagged preconditioners

The hierarchy staleness detector of Section 10.6 compares conjugate-gradient iteration counts across solves. Under a forcing term those counts are taken at *different* tolerances and are not comparable as raw numbers.

Simply ignoring loosened solves does not work, and the reason is guard (6.4): it can raise η enough that a run converges without ever solving at the deck’s tolerance, so a detector that only counted full-tolerance solves would never fire at all. Since that detector is the only rebuild trigger on the quasi-static path, the hierarchy would be built once at the reference configuration and kept for an entire load history — precisely the failure Section 10.2 exists to prevent.

Carina instead rescales. Because a Krylov residual falls geometrically, the iteration count is roughly proportional to the number of digits of reduction requested, so

$$\hat{m}_k = m_k \cdot \frac{\log \varepsilon_{\text{lin}}}{\log \eta_k} \quad (6.6)$$

converts a count m_k taken at tolerance η_k into the count the same solve would have cost at the deck’s tolerance. At $\eta_k = \varepsilon_{\text{lin}}$ this is exactly the identity, so a run with no forcing term feeds the detector precisely the numbers it always did. The estimate is coarse, but the detector’s threshold — a factor of three — is coarser.

Chapter 7

Quasi-Newton and gradient methods

Implementation: `src/nonlinear_solvers.jl`, `src/linear_solvers.jl`

Three alternatives to Newton are implemented, all matrix-free in the sense that none forms or factors a tangent. They exist for cases where assembling $\mathbf{K}$ is unattractive, and each has a regime where it is the right choice and a regime where it fails.

7.1 L-BFGS

Limited-memory BFGS builds an implicit approximation to $\mathbf{K}^{-1}$ from the last m secant pairs

$$\mathbf{s}_k = \mathbf{u}_{k+1} - \mathbf{u}_k, \quad \mathbf{y}_k = \mathbf{R}_k - \mathbf{R}_{k+1}, \quad \rho_k = \frac{1}{\mathbf{y}_k^\top \mathbf{s}_k}, \quad (7.1)$$

and applies it through the two-loop recursion of Algorithm 3, at $O(mn)$ per iteration with no matrix at all.

Algorithm 3 L-BFGS two-loop recursion

Input: residual $\mathbf{R}_k$, history $\{\mathbf{s}_i, \mathbf{y}_i, \rho_i\}$ newest first, initial inverse Hessian $\mathbf{H}_0$

Output: descent direction $\mathbf{d}$

```

1:  $\mathbf{q} \leftarrow \mathbf{R}_k$ 
2: for  $i = 1$  to  $m$  do  $\triangleright$  newest to oldest
3:    $a_i \leftarrow \rho_i \mathbf{s}_i^\top \mathbf{q}, \quad \mathbf{q} \leftarrow \mathbf{q} - a_i \mathbf{y}_i$ 
4: end for
5:  $\mathbf{d} \leftarrow \mathbf{H}_0 \mathbf{q}$   $\triangleright$  Barzilai–Borwein, or Jacobi with no history
6: for  $i = m$  down to  $1$  do  $\triangleright$  oldest to newest
7:    $b_i \leftarrow \rho_i \mathbf{y}_i^\top \mathbf{d}, \quad \mathbf{d} \leftarrow \mathbf{d} + (a_i - b_i) \mathbf{s}_i$ 
8: end for
9: return  $\mathbf{d}$ 

```

A pair is admitted to the history only when $\mathbf{y}_i^\top \mathbf{s}_i > 0$. That curvature condition is what keeps ρ_i finite and the implied inverse Hessian positive definite, hence $\mathbf{d}$ a descent direction; a pair failing it is discarded rather than stored.

The choice of $\mathbf{H}_0$ matters more here than the textbook presentation suggests. Carina uses

Barzilai–Borwein scaling from the most recent pair,

$$\mathbf{H}_0 = \gamma_0 \mathbf{I}, \quad \gamma_0 = \frac{\mathbf{s}_k^\top \mathbf{y}_k}{\mathbf{y}_k^\top \mathbf{y}_k}, \quad (7.2)$$

once history exists, and a Jacobi diagonal before then. The Jacobi fallback is not cosmetic: on the first step of an implicit dynamic problem there is no history, and $c_M = 1/(\beta \Delta t^2)$ can be enormous (equation 4.5), so $\mathbf{H}_0 = \mathbf{I}$ would produce a first step wrong by many orders of magnitude.

Where it works, and where it does not. L-BFGS is the fastest option Carina has for implicit *dynamics* on GPU, and it fails outright on quasi-static problems — stalling roughly seven orders short of tolerance, on CPU and GPU alike, at any history size.

Section 4.2 explains why. At small Δt the effective operator $\mathbf{K}_{\text{eff}} = \mathbf{K} + c_M \mathbf{M}$ is mass-dominated and strongly diagonally dominant, which is exactly the structure a rank- m update models well. Quasi-static has no mass term: the operator is the bare stiffness, whose spectrum spans the mesh-refinement range, and a rank-10 model cannot represent it. This is a statement about the operator, not about the implementation or the device — which is why no amount of history size rescues it.

7.2 Nonlinear conjugate gradient

Preconditioned Polak–Ribière⁺. With $\mathbf{g}_k = \mathbf{M} \mathbf{R}_k$ the preconditioned gradient,

$$\beta_k = \max \left(0, \frac{\mathbf{R}_k^\top \mathbf{g}_k - \mathbf{R}_k^\top \mathbf{g}_{k-1}}{\mathbf{R}_{k-1}^\top \mathbf{g}_{k-1}} \right), \quad \mathbf{d}_k = \mathbf{g}_k + \beta_k \mathbf{d}_{k-1}. \quad (7.3)$$

The clamp at zero is the “+” variant and amounts to restarting with a steepest-descent direction whenever the formula would turn negative, which guarantees $\mathbf{d}_k$ remains a descent direction. Carina restarts additionally when successive gradients lose orthogonality beyond a tolerance, and optionally at a fixed interval.

Only Jacobi preconditioning is available at this level; asking for anything stronger is rejected rather than silently downgraded, since a preconditioner that silently did nothing would present as a mysterious convergence problem.

7.3 Preconditioned steepest descent

The direction is $\mathbf{d}_k = \mathbf{M} \mathbf{R}_k$, with an Armijo line search on the total potential energy rather than on the residual norm:

$$\Pi(\mathbf{u} + \alpha \mathbf{d}) \leq \Pi(\mathbf{u}) + c \alpha \nabla \Pi \cdot \mathbf{d}, \quad \nabla \Pi \cdot \mathbf{d} = -\mathbf{R}^\top \mathbf{d}. \quad (7.4)$$

Using the energy is available here because the directional derivative is exact and cheap, and Π is the quantity the equilibrium problem actually minimizes. This is the slowest and most robust option: linear convergence, but it makes progress on problems where the others stall.

Chapter 8

Krylov methods

Implementation: src/linear_solvers.jl

8.1 Why conjugate gradients

Every operator Carina hands to a linear solver is symmetric positive definite: the tangent stiffness of a hyperelastic material is the Hessian of a stored energy, and the Newmark effective operator (4.6) adds $c_M \mathbf{M}$ with $c_M > 0$, which can only improve matters. Conjugate gradients is therefore the method of choice, and the input parser maps every iterative alias — `cg`, `krylov`, `minres` — onto it.

The assumption is checked rather than trusted. Carina measures $\|\mathbf{K} - \mathbf{K}^\top\|_\infty / \|\mathbf{K}\|_\infty$ once per run and reports if the tangent is not symmetric to roundoff, since CG applied to a non-symmetric operator does not merely converge slowly — it converges to the wrong answer, silently. The measured value on the assembled path is $\sim 10^{-16}$.

8.2 Preconditioned conjugate gradients

With a symmetric positive definite preconditioner $\mathbf{M} \approx \mathbf{K}^{-1}$:

One operator application and one preconditioner application per iteration, and no other access to $\mathbf{K}$ — which is what makes the matrix-free operators of Chapter 11 usable at all.

The error bound

$$\|\mathbf{e}_i\|_{\mathbf{K}} \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^i \|\mathbf{e}_0\|_{\mathbf{K}}, \quad \kappa = \kappa(\mathbf{M}\mathbf{K}), \quad (8.1)$$

is what every preconditioner in Chapter 9 exists to improve, and it is also the justification for the rescaling (6.6): (8.1) is geometric in i , so iterations are proportional to digits of reduction requested.

Stopping. Carina terminates on the preconditioned residual $\mathbf{M}$ -norm relative to its own initial value, $\sqrt{\rho_i} \leq \eta \sqrt{\rho_0}$, because $\rho_i = \mathbf{r}_i^\top \mathbf{M} \mathbf{r}_i$ is already computed each iteration and needs no extra inner product. The consequence for the forcing-term theory is noted in Chapter 6.

Algorithm 4 Preconditioned conjugate gradients

Input: operator $\mathbf{K}$, right-hand side $\mathbf{b}$, preconditioner $\mathbf{M}$, forcing term η , iteration cap $i_{\max}$
 $\triangleright$ cold start; see below

```

1:  $\mathbf{x} \leftarrow \mathbf{0}$ 
2:  $\mathbf{r} \leftarrow \mathbf{b}$ ,  $\mathbf{z} \leftarrow \mathbf{M}\mathbf{r}$ ,  $\mathbf{p} \leftarrow \mathbf{z}$ ,  $\rho \leftarrow \mathbf{r}^\top \mathbf{z}$ 
3:  $\rho_0 \leftarrow \rho$ 
4: for  $i = 1$  to  $i_{\max}$  do
5:    $\mathbf{q} \leftarrow \mathbf{K}\mathbf{p}$   $\triangleright$  the only operator application
6:    $\alpha \leftarrow \rho / (\mathbf{p}^\top \mathbf{q})$ 
7:    $\mathbf{x} \leftarrow \mathbf{x} + \alpha \mathbf{p}$ ,  $\mathbf{r} \leftarrow \mathbf{r} - \alpha \mathbf{q}$ 
8:    $\mathbf{z} \leftarrow \mathbf{M}\mathbf{r}$   $\triangleright$  the only preconditioner application
9:    $\rho^{\text{new}} \leftarrow \mathbf{r}^\top \mathbf{z}$ 
10:  if  $\sqrt{\rho^{\text{new}}} \leq \eta \sqrt{\rho_0}$  then
11:    return  $\mathbf{x}$   $\triangleright$  converged in the  $\mathbf{M}$ -norm
12:  end if
13:   $\mathbf{p} \leftarrow \mathbf{z} + (\rho^{\text{new}} / \rho) \mathbf{p}$ ,  $\rho \leftarrow \rho^{\text{new}}$ 
14: end for
```

Cold start. The workspace solution is zeroed before every solve rather than warm-started from the previous Newton iteration. A stale warm-start is not merely unhelpful here: the operator has changed between Newton iterations, so the previous solution is the answer to a different question, and beginning from it can leave the residual larger than beginning from zero.

8.3 Device-resident conjugate gradients

On GPU the entire iteration runs on the device. This is not a detail of implementation but a change in what dominates the cost: a host-resident CG must transfer ρ_i and α_i back to the host each iteration to make the scalar decisions, and each transfer is a synchronization point that idles the device for longer than the arithmetic it guards.

The dot products are reduced on-device by a two-stage tree — a fixed number of workgroups each reducing a strided slice, then a single workgroup reducing the partials — and the scalars stay in device memory. The only quantity that must reach the host is the convergence flag.

8.4 Direct solvers

A sparse $LDL^\top$ factorization remains the right choice for small to medium problems on CPU: it needs no tuning and cannot fail to converge. It is unavailable on GPU, and asking for it there is a hard error rather than a silent fallback.

One hybrid exists. For a linear-elastic model the stiffness is constant, so Carina factors $\mathbf{K}$ once on the host, uploads the Cholesky factor to the device, and thereafter solves by two device triangular solves. This gives direct-solver robustness with no host transfer per step, and applies only because linearity makes the factorization reusable for the entire run.

Chapter 9

Preconditioners

Implementation: `src/linear_solvers.jl`

A preconditioner is an approximation $\mathbf{M} \approx \mathbf{K}^{-1}$ that is cheap to apply and reduces $\kappa(\mathbf{MK})$ in (8.1). It must be symmetric positive definite for conjugate gradients to remain valid. It need not be accurate: Remark 3 applies here too — the preconditioner shapes the path, never the fixed point.

9.1 Jacobi

$\mathbf{M} = \mathbf{D}^{-1}$ with $\mathbf{D} = \text{diag}(\mathbf{K})$. One vector scaling per application, trivially parallel, always available. It removes the conditioning that comes from heterogeneous element sizes and material properties, and nothing else — in particular it does nothing about the $(L/h)^2$ growth with mesh refinement that dominates quasi-static problems.

Its unusual effectiveness in implicit dynamics at small Δt is a direct consequence of (4.5): when $c_M \mathbf{M}$ dominates $\mathbf{K}_{\text{eff}}$, and $\mathbf{M}$ is nearly diagonal for the element types in use, the diagonal captures most of the operator.

9.2 Incomplete LDL^T

An incomplete factorization $\mathbf{K} \approx \mathbf{LDL}^T$ that drops fill outside a chosen sparsity pattern, applied by two triangular solves. Substantially stronger than Jacobi for ill-conditioned systems, and the natural choice for J_2 plasticity on non-uniform meshes.

It is CPU-only, and the obstruction is structural rather than a missing port: a triangular solve is inherently sequential along each dependency chain, which is the one access pattern a GPU cannot exploit. This is why the GPU path needs either a polynomial preconditioner or multigrid, both of which reduce to matrix–vector products.

9.3 Chebyshev polynomial preconditioner

A polynomial preconditioner approximates $\mathbf{K}^{-1}$ by a polynomial in $\mathbf{K}$, so applying it costs only matrix–vector products — which makes it GPU-friendly where triangular solves are not.

Carina implements Chebyshev polynomials of the *fourth kind* with optimal weights, following Lottes [3] as implemented in Ifpack2. This variant has three properties that matter here:

- It requires only $\lambda_{\max}$, not the eigenvalue *ratio*. Standard Chebyshev needs a lower bound $\lambda_{\min}$ as well, and a poor estimate of it degrades the polynomial badly; $\lambda_{\min}$ is also the harder of the two to estimate.
- Jacobi scaling is baked into the recurrence, so $\mathbf{D}^{-1}$ is applied at each stage rather than as a separate wrapper.
- The optimal weights make the resulting operator symmetric positive definite *by construction*, so it is admissible as a CG preconditioner without the squaring trick — costing k matrix–vector products per application instead of $2k$.

With $\sigma = 1/\lambda_{\max}$ and weights β_i tabulated for degrees 1 through 16:

Algorithm 5 Fourth-kind Chebyshev preconditioner application, degree k

Input: $\mathbf{b}$, operator $\mathbf{K}$, $\mathbf{D}^{-1}$, $\sigma = 1/\lambda_{\max}$, weights $\beta_1, \dots, \beta_k$

Output: $\mathbf{y} \approx \mathbf{K}^{-1}\mathbf{b}$

```

1:  $\mathbf{z} \leftarrow \frac{4}{3}\sigma \mathbf{D}^{-1}\mathbf{b}$ ,  $\mathbf{x} \leftarrow \mathbf{z}$ ,  $\mathbf{y} \leftarrow \beta_1\mathbf{z}$ 
2: for  $i = 1$  to  $k - 1$  do
3:    $\gamma_i \leftarrow \frac{2i-1}{2i+3}$ ,  $\rho_i \leftarrow \frac{8i+4}{2i+3}\sigma$ 
4:    $\mathbf{z} \leftarrow \rho_i \mathbf{D}^{-1}(\mathbf{b} - \mathbf{K}\mathbf{x}) + \gamma_i\mathbf{z}$ 
5:    $\mathbf{x} \leftarrow \mathbf{x} + \mathbf{z}$ 
6:    $\mathbf{y} \leftarrow \mathbf{y} + \beta_{i+1}\mathbf{z}$ 
7: end for
8: return  $\mathbf{y}$ 
```

▷ one operator application
 ▷ raw fourth-kind iterate
 ▷ weighted accumulation

Note that $\mathbf{x}$ and $\mathbf{y}$ are distinct: $\mathbf{x}$ carries the unweighted recurrence that generates the polynomial basis, while $\mathbf{y}$ accumulates the optimally weighted combination that is actually returned. Collapsing them — returning $\mathbf{x}$ — gives the ordinary fourth-kind polynomial without the optimal weights, and loses the positive-definiteness that makes the operator admissible to conjugate gradients.

Spectral bound. $\lambda_{\max}$ of $\mathbf{D}^{-1}\mathbf{K}$ is estimated by ten power iterations followed by a Rayleigh quotient, then inflated by a safety factor of 1.1. The inflation is deliberate and one-sided: overestimating $\lambda_{\max}$ makes the polynomial conservative and merely slower, whereas underestimating it puts part of the spectrum outside the interval the polynomial damps, and the preconditioner amplifies those modes instead of removing them.

Symmetric scaling. On the assembled path the polynomial is built on $\mathbf{D}^{-1/2}\mathbf{K}\mathbf{D}^{-1/2}$ rather than on $\mathbf{K}$, for the reason given in Section 2.4: penalty-scale entries give $\mathbf{K}$ eigenvalues of order 10^{15} while the symmetrically scaled operator has eigenvalues of order one, and a polynomial fitted to the former is useless. The preconditioner is then $\mathbf{M} = \mathbf{D}^{-1/2}p_k(\mathbf{S})\mathbf{D}^{-1/2}$, which is symmetric positive definite whenever p_k is.

Chapter 10

Smoothed-aggregation multigrid

Implementation: `src/gpu_amg.jl`, `src/linear_solvers.jl`

10.1 Why multigrid

Every preconditioner of Chapter 9 is local: it couples each unknown to its immediate algebraic neighbors. Local operators damp high-frequency error efficiently and low-frequency error not at all, which is exactly why $\kappa(\mathbf{K})$ grows like $(L/h)^2$ under refinement — the smoothest error modes are the ones a local operator cannot see.

Multigrid removes what smoothing cannot by representing the smooth error on a coarser problem, where it is no longer smooth relative to the grid. The payoff is an iteration count that is nearly *independent* of problem size. On a 530k-DOF torsion problem, algebraic multigrid holds 5–17 conjugate gradient iterations across a range of Δt over which Jacobi grows from 30 to 442.

Carina uses *algebraic* multigrid: the hierarchy is built from the matrix alone, with no geometric coarse mesh, which is what makes it applicable to unstructured meshes.

10.2 The near-nullspace

Algebraic multigrid needs to know which error modes the smoother will not remove, so that the coarse space can represent them. For elasticity these are the modes with no strain energy: the six rigid-body modes — three translations and three rotations. They form the columns of the *near-nullspace* $\mathbf{B} \in \mathbb{R}^{3n_{\text{node}} \times 6}$, built from nodal coordinates about the centroid $\bar{\mathbf{x}}$:

$$\mathbf{B} = \left[\begin{array}{ccc|ccc} 1 & 0 & 0 & 0 & \hat{z} & -\hat{y} \\ 0 & 1 & 0 & -\hat{z} & 0 & \hat{x} \\ 0 & 0 & 1 & \hat{y} & -\hat{x} & 0 \end{array} \right], \quad \hat{\mathbf{x}} = \mathbf{x} - \bar{\mathbf{x}}, \quad (10.1)$$

one such block per node, restricted to the free degrees of freedom.

Current configuration, not reference. Carina builds $\mathbf{B}$ from the *current* nodal coordinates $\mathbf{X} + \mathbf{u}$ at every hierarchy build, not from the reference configuration. At finite deformation the true near-nullspace of $\mathbf{K}(\mathbf{u})$ consists of rigid-body motions about the *deformed* body; a nullspace frozen at the reference configuration describes rotations the body no longer has. The

consequence is not subtle: on a twisted-bar run, a frozen reference nullspace degraded the iteration count from 63 to over 400 as the bar rotated, while the hierarchy build count stayed at one and nothing in the log indicated a problem.

10.3 Hierarchy construction

Each level is built in four stages.

1. Strength of connection. A symmetric strength measure filters $\mathbf{K}$ into a graph $\mathbf{S}$ retaining only algebraically strong couplings, so that aggregation follows the directions in which error actually propagates.

2. Aggregation. Standard aggregation partitions the nodes of $\mathbf{S}$ into disjoint aggregates, each becoming one coarse node.

3. Tentative prolongator. For each aggregate g , the restriction of the near-nullspace to that aggregate's rows is factored,

$$\mathbf{B}|_g = \mathbf{Q}_g \mathbf{R}_g, \quad (10.2)$$

by a thin QR. The orthonormal $\mathbf{Q}_g$ becomes the aggregate's block of the tentative prolongator $\mathbf{T}$, and $\mathbf{R}_g$ becomes the coarse near-nullspace $\mathbf{B}_c$ for the next level. This construction guarantees

$$\mathbf{B} = \mathbf{T} \mathbf{B}_c, \quad (10.3)$$

i.e. the coarse space represents every near-null mode *exactly* — which is the whole point of supplying $\mathbf{B}$.

Remark 10.1. An aggregate with fewer nodes than $\mathbf{B}$ has columns is rank deficient, and its QR yields fewer than six usable columns. The implementation handles this by giving such an aggregate $r = \min(|g|, 6)$ columns and leaving the remainder structurally empty, rather than producing near-zero columns that would inflate every downstream sparse product.

4. Prolongator smoothing. The tentative prolongator is improved by one damped Jacobi sweep,

$$\mathbf{P} = \left(\mathbf{I} - \frac{4}{3\lambda_{\max}} \mathbf{D}^{-1} \mathbf{K} \right) \mathbf{T}, \quad (10.4)$$

which lowers the energy of the coarse basis functions and is what distinguishes *smoothed* aggregation from plain aggregation. Restriction is $\mathbf{R} = \mathbf{P}^\top$, and the coarse operator is the Galerkin triple product

$$\mathbf{K}_c = \mathbf{R} \mathbf{K} \mathbf{P}. \quad (10.5)$$

Remark 10.2 (Memory). Evaluated naively, (10.5) preallocates from an up-front nonzero-count estimate that is wildly pessimistic for a six-column near-nullspace — demanding tens of gigabytes at 1.5M DOF for a product whose true size is about 4 GB. Carina evaluates the fine-level product in column slabs of $\mathbf{P}$, so peak transient memory is one slab's product rather than the whole estimate, and hands the much smaller coarse problem to the standard routine for the remaining levels.

10.4 The V-cycle

One $V(\nu, \nu)$ -cycle applied to a residual $\mathbf{r}$ produces a correction $\mathbf{z}$:

Algorithm 6 $\text{VCYCLE}(\ell, \mathbf{b}^\ell)$ — returns a correction $\mathbf{x}^\ell$

```

1: function  $\text{VCYCLE}(\ell, \mathbf{b}^\ell)$ 
2:   if  $\ell$  is the coarsest level then
3:     return  $\mathbf{K}_\ell^+ \mathbf{b}^\ell$   $\triangleright$  dense pseudo-inverse, formed at build time
4:   end if
5:    $\mathbf{x} \leftarrow \mathbf{0}$ 
6:   for  $i = 1$  to  $\nu$  do  $\triangleright$  pre-smooth
7:      $\mathbf{x} \leftarrow \mathbf{x} + \omega_\ell \mathbf{D}_\ell^{-1} (\mathbf{b}^\ell - \mathbf{K}^\ell \mathbf{x})$ 
8:   end for
9:    $\mathbf{r} \leftarrow \mathbf{b}^\ell - \mathbf{K}^\ell \mathbf{x}$ 
10:   $\mathbf{b}^{\ell+1} \leftarrow \mathbf{R}^\ell \mathbf{r}$   $\triangleright$  restrict
11:   $\mathbf{x} \leftarrow \mathbf{x} + \mathbf{P}^\ell \text{VCYCLE}(\ell + 1, \mathbf{b}^{\ell+1})$   $\triangleright$  coarse correction
12:  for  $i = 1$  to  $\nu$  do  $\triangleright$  post-smooth
13:     $\mathbf{x} \leftarrow \mathbf{x} + \omega_\ell \mathbf{D}_\ell^{-1} (\mathbf{b}^\ell - \mathbf{K}^\ell \mathbf{x})$ 
14:  end for
15:  return  $\mathbf{x}$ 
16: end function

```

The smoother is damped Jacobi with $\omega_\ell = 4/(3\lambda_{\max}^\ell)$, the same weight used for prolongator smoothing, with $\lambda_{\max}^\ell$ estimated per level by power iteration. The coarsest level is solved by a dense pseudo-inverse, formed once at build time; at that size a dense solve is cheaper than another level of hierarchy.

The cycle is entered as $\text{VCYCLE}(1, \mathbf{r})$ with $\mathbf{r}$ the conjugate-gradient residual, and the correction it returns is the preconditioner application $\mathbf{M}\mathbf{r}$ of Section 8.2.

Symmetry of the cycle — equal pre- and post-smoothing, and $\mathbf{R} = \mathbf{P}^\top$ — is what makes the resulting operator symmetric, hence admissible as a conjugate-gradient preconditioner.

10.5 GPU execution

The hierarchy is built on the host, where the sparse pattern already lives, and converted to device CSR. The cycle then runs entirely on the device. The fine level is special: rather than storing $\mathbf{K}$ on the device, its matrix-vector products go through the matrix-free action of Chapter 11, so the fine matrix is never formed there at all. Only the coarse levels — orders of magnitude smaller — are stored explicitly.

10.6 Lagging and staleness

Hierarchy setup costs seconds at 500k DOF, far more than a single V-cycle application, so the hierarchy is *lagged*: built once and reused across Newton iterations and time steps. Reuse is safe because the hierarchy is a preconditioner — an out-of-date one costs iterations, never accuracy.

Two triggers force a rebuild. A change in c_M means the operator itself has changed (a new Δt). And a staleness detector watches the conjugate-gradient iteration count: it latches a baseline on the first solve after a build and requests a rebuild when a later count exceeds three times that baseline, subject to a floor so that small counts do not thrash.

On the quasi-static path this detector is the *only* rebuild trigger, because $c_M = 0$ there and the first test never fires. That is what makes the interaction with forcing terms consequential, and it is treated in Section 6.4's neighboring discussion at the end of Chapter 6.

10.7 When multigrid is the right tool

AMG targets operators whose difficulty is the smooth end of the spectrum: quasi-static problems and moderate- Δt dynamics. It is not universally better, and two limits are worth naming.

At small Δt in implicit dynamics, $\mathbf{K}_{\text{eff}}$ is mass-dominated (Section 4.2) and already well conditioned. There is little smooth error for a coarse grid to remove, and measurements on two GPU architectures confirm the V-cycle does not repay its per-application cost: Jacobi is faster.

At very large Δt on violently dynamic problems, the Newmark predictor can overshoot into near-inverted configurations whose tangent is indefinite. Conjugate gradients is then invalid regardless of preconditioner, and explicit integration is the appropriate method.

Chapter 11

Matrix-free operators

Implementation: `src/physics.jl`, `src/two_phase_action.jl`

11.1 The action, not the matrix

Conjugate gradients never needs $\mathbf{K}$ — only its action $\mathbf{K}\mathbf{v}$ on a sequence of vectors. A matrix-free operator supplies that action by assembling directly:

$$(\mathbf{K}\mathbf{v})_a = \int_{\Omega_0} \nabla N_a \cdot \mathbb{A} \cdot \nabla N_b \, dV \, v_b = \int_{\Omega_0} \nabla N_a : \delta \mathbf{P}(\nabla \mathbf{v}) \, dV, \quad (11.1)$$

where $\delta \mathbf{P}$ is the directional derivative of the stress in the direction $\nabla \mathbf{v}$ — equation (3.4) for neo-Hookean. The right-hand form is the one that matters: the tangent’s action on *one* direction never requires the 81-component $\mathbb{A}$ to be formed, so the closed-form derivative costs a few 3×3 products per quadrature point.

Three things follow. Memory drops from $O(\text{nnz})$ to $O(n)$ — measured at 5.83 GB down to 0.245 GB on a 530k-DOF problem, which is the difference between fitting on a GPU and not. There is no assembly phase and no sparse pattern to build. And the operator is exactly as current as the displacement it is evaluated at, so no staleness question arises.

The cost is that each application redoes the constitutive work. Whether that is a good trade depends on the arithmetic intensity of the element kernel against the bandwidth of storing and re-reading a sparse matrix; on GPU it is decisively favorable.

11.2 Operator fusion

The Newmark system needs $\mathbf{K}_{\text{eff}}\mathbf{v} = \mathbf{K}\mathbf{v} + c_M \mathbf{M}\mathbf{v}$. Computed as two separate assemblies, the element loop runs twice and each pass gathers the same nodal data. Fusing them into a single kernel that accumulates both contributions per quadrature point halves the gather traffic, and the gather is what the operator is bound by.

The same reasoning produces a diagonal-only kernel for the Jacobi preconditioner: $\text{diag}(\mathbf{K}_{\text{eff}})$ is assembled directly rather than extracted from an operator that is never formed.

11.3 Where the time goes

Profiling the fused Newmark action on an A100 at 530k DOF attributes 67% of device time to the stiffness matrix–vector product, with the device busy 72% of the step. The step, to a good approximation, *is* the matvec.

That single fact organizes the optimization work. It is why atomic contention is not the bottleneck — ablating the atomics changes nothing measurable — and why the loss ordering across architectures tracks effective cache size rather than arithmetic throughput. It is also why the forcing terms of Chapter 6 pay so much better on the GPU paths than on assembled CPU runs: on this path, removing a conjugate-gradient iteration removes almost a whole iteration’s worth of wall time.

11.4 Two-phase assembly

The default kernel assigns one thread per element: gather the element’s nodal values, loop the quadrature points serially, scatter the results with atomic adds. The working set held across that quadrature loop is what limits occupancy.

The two-phase form splits it. Phase one assigns one thread per (element, quadrature point) pair, writing each contribution to a disjoint slot of a staging array — eight times the threads, an eighth of the per-thread state, and no atomics. Phase two assigns one thread per node and sums the staging slots incident on it through an inverse adjacency built once on the host, again without atomics.

The transformation is sound and it is implemented, but measurement did not support it: 0.66 to 0.80 times the throughput of the one-thread-per-element kernel on all three architectures tested. The extra staging traffic exceeds what the occupancy gain returns. It is documented here because the reasoning is correct and the conclusion is specific to the current data layout — an element-blocked field layout could change it.

11.5 Mixed precision

The V-cycle of Chapter 10 applies the matrix-free action five times per call at $\nu = 2$, which on a consumer GPU — where double precision runs at 1/32 rate — dominates the iteration. Carina evaluates the fine-level *smoother* action in single precision while the operator conjugate gradients solves against stays in double.

This is legitimate, and the argument is the one from Remark 3. A preconditioner may be any symmetric positive definite approximation; reducing its precision changes which approximation it is, not what the iteration converges to. What would *not* be legitimate is reducing the precision of the operator itself, which would change the fixed point. The reduced-precision V-cycle also remains a fixed linear operator across applications, so ordinary conjugate gradients — rather than a flexible variant — stays valid.

The gate is a property of the model, not a user request: Carina enables the reduced-precision path only for models that demonstrably narrow to single precision, and verifies this rather than assuming it. A model whose action silently falls back to double would otherwise present as a preconditioner that was never actually accelerated.

Bibliography

- [1] Stanley C. Eisenstat and Homer F. Walker. Choosing the forcing terms in an inexact Newton method. *SIAM Journal on Scientific Computing*, 17(1):16–32, 1996.
- [2] C. T. Kelley. *Iterative Methods for Linear and Nonlinear Equations*. Frontiers in Applied Mathematics. SIAM, 1995.
- [3] James Lottes. Optimal polynomial smoothers for multigrid V-cycles. *Numerical Linear Algebra with Applications*, 30(6), 2023. arXiv:2202.08830.